

3 让研究人员绞尽脑汁的Transformer位置编码

Feb By 苏剑林 | 2021-02-03 | 240829位读者

不同于RNN、CNN等模型，对于Transformer模型来说，位置编码的加入是必不可少的，因为纯粹的Attention模块是无法捕捉输入顺序的，即无法区分不同位置的Token。为此我们大体有两个选择：1、想办法将位置信息融入到输入中，这构成了绝对位置编码的一般做法；2、想办法微调一下Attention结构，使得它有能力分辨不同位置的Token，这构成了相对位置编码的一般做法。

虽然说起来主要就是绝对位置编码和相对位置编码两大类，但每一类其实又能衍生出各种各样的变种，为此研究人员可算是煞费苦心、绞尽脑汁了，此外还有一些不按套路出牌的位置编码。本文就让我们来欣赏一下研究人员为了更好地表达位置信息所构建出来的“八仙过海，各显神通”般的编码方案。

绝对位置编码

形式上来看，绝对位置编码是相对简单的一种方案，但即便如此，也不妨碍各路研究人员的奇思妙想，也有不少变种。一般来说，绝对位置编码会加到输入中：在输入的第 k 个向量 $\mathbf{x}_k$ 中加入位置向量 $\mathbf{p}_k$ 变为 $\mathbf{x}_k + \mathbf{p}_k$ ，其中 $\mathbf{p}_k$ 只依赖于位置编号 k 。

训练式

很显然，绝对位置编码的一个最朴素方案是不特意去设计什么，而是直接将位置编码当作可训练参数，比如最大长度为512，编码维度为768，那么就初始化一个 512×768 的矩阵作为位置向量，让它随着训练过程更新。现在的BERT、GPT等模型所用的就是这种位置编码，事实上它还可以追溯得更早，比如2017年Facebook的《Convolutional Sequence to Sequence Learning》就已经用到了它。

对于这种训练式的绝对位置编码，一般的认为它的缺点是没有外推性，即如果预训练最大长度为512的话，那么最多就只能处理长度为512的句子，再长就处理不了了。当然，也可以将超过512的位置向量随机初始化，然后继续微调。但笔者最近的研究表明，通过层次分解的方式，可以使得绝对位置编码能外推到足够长的范围，同时保持还不错的效果，细节请参考笔者之前的博文《层次分解位置编码，让BERT可以处理超长文本》。因此，其实外推性也不是绝对位置编码的明显缺点。

三角式

三角函数式位置编码，一般也称为Sinusoidal位置编码，是Google的论文《Attention is All You Need》所提出来的一个显式解：

$$\begin{cases} \mathbf{p}_{k,2i} = \sin\left(k/10000^{2i/d}\right) \\ \mathbf{p}_{k,2i+1} = \cos\left(k/10000^{2i/d}\right) \end{cases} \quad (1)$$

其中 $\mathbf{p}_{k,2i}$, $\mathbf{p}_{k,2i+1}$ 分别是位置 k 的编码向量的第 $2i$, $2i+1$ 个分量， d 是位置向量的维度。

很明显，三角函数式位置编码的特点是有显式的生成规律，因此可以期望于它有一定的外推性。另外一个使用它的理由是：由于 $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$ 以及 $\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$ ，这表

明位置 $\alpha + \beta$ 的向量可以表示成位置 α 和位置 β 的向量组合，这提供了表达相对位置信息的可能性。但很奇怪的是，现在我们很少能看到直接使用这种形式的绝对位置编码的工作，原因不详。

递归式

原则上来说，RNN模型不需要位置编码，它在结构上就自带了学习到位置信息的可能性（因为递归就意味着我们可以训练一个“数数”模型），因此，如果在输入后面先接一层RNN，然后再接Transformer，那么理论上就不需要加位置编码了。同理，我们也可以用RNN模型来学习一种绝对位置编码，比如从一个向量 p_0 出发，通过递归格式 $p_{k+1} = f(p_k)$ 来得到各个位置的编码向量。

ICML 2020的论文《Learning to Encode Position for Transformer with Continuous Dynamical Model》把这个思想推到了极致，它提出了用微分方程（ODE） $dp_t/dt = h(p_t, t)$ 的方式来建模位置编码，该方案称之为FLOATER。显然，FLOATER也属于递归模型，函数 $h(p_t, t)$ 可以通过神经网络来建模，因此这种微分方程也称为神经微分方程，关于它的工作最近也逐渐多了起来。

理论上来说，基于递归模型的位置编码也具有比较好的外推性，同时它也比三角函数式的位置编码有更好的灵活性（比如容易证明三角函数式的位置编码就是FLOATER的某个特解）。但是很明显，递归形式的位置编码牺牲了一定的并行性，可能会带速度瓶颈。

相乘式

刚才我们说到，输入 x_k 与绝对位置编码 p_k 的组合方式一般是 $x_k + p_k$ ，那有没有“不一般”的组合方式呢？比如 $x_k \otimes p_k$ （逐位相乘）？我们平时在搭建模型的时候，对于融合两个向量有多种方式，相加、相乘甚至拼接都是可以考虑的，怎么大家在做绝对位置编码的时候，都默认只考虑相加了？

很抱歉，笔者也不知道答案。可能大家默认选择相加是因为向量的相加具有比较鲜明的几何意义，但是对于深度学习模型来说，这种几何意义其实没有什么实际的价值。最近笔者看到的一个实验显示，似乎将“加”换成“乘”，也就是 $x_k \otimes p_k$ 的方式，似乎比 $x_k + p_k$ 能取得更好的结果。具体效果笔者也没有完整对比过，只是提供一种可能性。关于实验来源，可以参考《中文语言模型研究：(1) 乘性位置编码》。

相对位置编码

相对位置并没有完整建模每个输入的位置信息，而是在算Attention的时候考虑当前位置与被Attention的位置的相对距离，由于自然语言一般更依赖于相对位置，所以相对位置编码通常也有着优秀的表现。对于相对位置编码来说，它的灵活性更大，更加体现出了研究人员的“天马行空”。

经典式

相对位置编码起源于Google的论文《Self-Attention with Relative Position Representations》，华为开源的NEZHA模型也用到了这种位置编码，后面各种相对位置编码变体基本也是依葫芦画瓢的简单修改。

一般认为，相对位置编码是由绝对位置编码启发而来，考虑一般的带绝对位置编码的Attention：

$$\begin{cases} \mathbf{q}_i = (\mathbf{x}_i + \mathbf{p}_i) \mathbf{W}_Q \\ \mathbf{k}_j = (\mathbf{x}_j + \mathbf{p}_j) \mathbf{W}_K \\ \mathbf{v}_j = (\mathbf{x}_j + \mathbf{p}_j) \mathbf{W}_V \\ a_{i,j} = \text{softmax}(\mathbf{q}_i \mathbf{k}_j^\top) \\ \mathbf{o}_i = \sum_j a_{i,j} \mathbf{v}_j \end{cases} \quad (2)$$

其中 softmax 对 j 那一维归一化，这里的向量都是指行向量。我们初步展开 $\mathbf{q}_i \mathbf{k}_j^\top$ ：

$$\mathbf{q}_i \mathbf{k}_j^\top = (\mathbf{x}_i + \mathbf{p}_i) \mathbf{W}_Q \mathbf{W}_K^\top (\mathbf{x}_j + \mathbf{p}_j)^\top = (\mathbf{x}_i \mathbf{W}_Q + \mathbf{p}_i \mathbf{W}_Q) (\mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{W}_K^\top \mathbf{p}_j^\top) \quad (3)$$

为了引入相对位置信息，Google把第一项位置去掉，第二项 $\mathbf{p}_j \mathbf{W}_K$ 改为二元位置向量 $\mathbf{R}_{i,j}^K$ ，变成

$$a_{i,j} = \text{softmax} \left(\mathbf{x}_i \mathbf{W}_Q (\mathbf{x}_j \mathbf{W}_K + \mathbf{R}_{i,j}^K)^\top \right) \quad (4)$$

以及 $\mathbf{o}_i = \sum_j a_{i,j} \mathbf{v}_j = \sum_j a_{i,j} (\mathbf{x}_j \mathbf{W}_V + \mathbf{p}_j \mathbf{W}_V)$ 中的 $\mathbf{p}_j \mathbf{W}_V$ 换成 $\mathbf{R}_{i,j}^V$ ：

$$\mathbf{o}_i = \sum_j a_{i,j} (\mathbf{x}_j \mathbf{W}_V + \mathbf{R}_{i,j}^V) \quad (5)$$

所谓相对位置，是将本来依赖于二元坐标 (i, j) 的向量 $\mathbf{R}_{i,j}^K, \mathbf{R}_{i,j}^V$ ，改为只依赖于相对距离 $i - j$ ，并且通常会进行截断，以适应不同任意的距离

$$\begin{aligned} \mathbf{R}_{i,j}^K &= \mathbf{p}_K [\text{clip}(i - j, p_{\min}, p_{\max})] \\ \mathbf{R}_{i,j}^V &= \mathbf{p}_V [\text{clip}(i - j, p_{\min}, p_{\max})] \end{aligned} \quad (6)$$

这样一来，只需要有限个位置编码，就可以表达出任意长度的相对位置（因为进行了截断），不管 $\mathbf{p}_K, \mathbf{p}_V$ 是选择可训练式的还是三角函数式的，都可以达到处理任意长度文本的需求。

XLNET式

XLNET式位置编码其实源自Transformer-XL的论文《Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context》，只不过因为使用了Transformer-XL架构的XLNET模型并在一定程度上超过了BERT后，Transformer-XL才算广为人知，因此这种位置编码通常也被冠以XLNET之名。

XLNET式位置编码源于对上述 $\mathbf{q}_i \mathbf{k}_j^\top$ 的完全展开：

$$\mathbf{q}_i \mathbf{k}_j^\top = \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{p}_j^\top + \mathbf{p}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{p}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{p}_j^\top \quad (7)$$

Transformer-XL的做法很简单，直接将 $\mathbf{p}_j$ 替换为相对位置向量 $\mathbf{R}_{i-j}$ ，至于两个 $\mathbf{p}_i$ ，则干脆替换为两个可训练的向量 $\mathbf{u}, \mathbf{v}$ ：

$$\mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{R}_{i-j}^\top + \mathbf{u} \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{v} \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{R}_{i-j}^\top \quad (8)$$

该编码方式中的 $\mathbf{R}_{i-j}$ 没有像式(6)那样进行截断，而是直接用了Sinusoidal式的生成方案，由于 $\mathbf{R}_{i-j}$ 的编码空间与 $\mathbf{x}_j$ 不一定相同，所以 $\mathbf{R}_{i-j}$ 前面的 $\mathbf{W}_K^\top$ 换了另一个独立的矩阵 $\mathbf{W}_{K,R}^\top$ ，还有 $\mathbf{u} \mathbf{W}_Q$ 、 $\mathbf{v} \mathbf{W}_Q$ 可以直接合并为

单个 $\mathbf{u}$ 、 $\mathbf{v}$ ，所以最终使用的式子是

$$\mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_{K,R}^\top \mathbf{R}_{i-j}^\top + \mathbf{u} \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{v} \mathbf{W}_{K,R}^\top \mathbf{R}_{i-j}^\top \quad (9)$$

此外， $\mathbf{v}_j$ 上的位置偏置就直接去掉了，即直接令 $\mathbf{o}_i = \sum_j a_{i,j} \mathbf{x}_j \mathbf{W}_V$ 。似乎从这个工作开始，后面的相对位置编码都只加到Attention矩阵上去，而不加到 $\mathbf{v}_j$ 上去了。

T5式

T5模型出自文章《Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer》，里边用到了一种更简单的相对位置编码。思路依然源自展开式(7)，如果非要分析每一项的含义，那么可以分别理解为“输入-输入”、“输入-位置”、“位置-输入”、“位置-位置”四项注意力的组合。如果我们认为输入信息与位置信息应该是独立（解耦）的，那么它们就不应该有过多的交互，所以“输入-位置”、“位置-输入”两项Attention可以删掉，而 $\mathbf{p}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{p}_j^\top$ 实际上只是一个只依赖于 (i, j) 的标量，我们可以直接将它作为参数训练出来，即简化为

$$\mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \beta_{i,j} \quad (10)$$

说白了，它仅仅是在Attention矩阵的基础上加一个可训练的偏置项而已，而跟XLNET式一样，在 $\mathbf{v}_j$ 上的位置偏置则直接被去掉了。包含同样的思想的还有微软在ICLR 2021的论文《Rethinking Positional Encoding in Language Pre-training》中提出的TUPE位置编码。

比较“别致”的是，不同于常规位置编码对将 $\beta_{i,j}$ 视为 $i-j$ 的函数并进行截断的做法，T5对相对位置进行了一个“分桶”处理，即相对位置是 $i-j$ 的位置实际上对应的是 $f(i-j)$ 位置，映射关系如下：

$i-j$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$f(i-j)$	0	1	2	3	4	5	6	7	8	8	8	8	9	9	9	9
$i-j$	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	...
$f(i-j)$	10	10	10	10	10	10	10	11	11	11	11	11	11	11	11	...

具体的映射代码，读者自行看源码就好。这个设计的思路其实也很直观，就是比较邻近的位置（0~7），我们需要比较得精细一些，所以给它们都分配一个独立的位置编码，至于稍远的位置（比如8~11），我们不用区分得太清楚，所以它们可以共用一个位置编码，距离越远，共用的范围就可以越大，直到达到指定范围再clip。

DeBERTa式

DeBERTa也是微软搞的，去年6月就发出来了，论文为《DeBERTa: Decoding-enhanced BERT with Disentangled Attention》，最近又小小地火了一把，一是因为它正式中了ICLR 2021，二则是它登上SuperGLUE的榜首，成绩稍微超过了T5。

其实DeBERTa的主要改进也是在位置编码上，同样还是从展开式(7)出发，T5是干脆去掉了第2、3项，只保留第4项并替换为相对位置编码，而DeBERTa则刚刚相反，它扔掉了第4项，保留第2、3项并且替换为相对位置编码（果然，科研就是枚举所有的排列组合看哪个最优）：

$$\mathbf{q}_i \mathbf{k}_j^\top = \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top + \mathbf{x}_i \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{R}_{i,j}^\top + \mathbf{R}_{j,i} \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{x}_j^\top \quad (11)$$

至于 $\mathbf{R}_{i,j}$ 的设计也是像式(6)那样进行截断的，没有特别的地方。

不过，DeBERTa比较有意思的地方，是提供了使用相对位置和绝对位置编码的一个新视角，它指出NLP的大多数任务可能都只需要相对位置信息，但确实有些场景下绝对位置信息更有帮助，于是它将整个模型分为两部分来理解。以Base版的MLM预训练模型为例，它一共有13层，前11层只是用相对位置编码，这部分称为Encoder，后面2层加入绝对位置信息，这部分它称之为Decoder，还弄了个简称EMD（Enhanced Mask Decoder）；至于下游任务的微调截断，则是使用前11层的Encoder加上1层的Decoder来进行。

SuperGLUE上的成绩肯定了DeBERTa的价值，但是它论文的各种命名真的是让人觉得极度不适，比如它自称的“Encoder”、“Decoder”就很容易让人误解这是一个Seq2Seq模型，比如EMD这个简称也跟Earth Mover's Distance重名。虽然有时候重名是不可避免的，但它重的名都是ML界大家都比较熟悉的对象，相当容易引起误解，真不知道作者是怎么想的...

其他位置编码

绝对位置编码和相对位置编码虽然花样百出，但仍然算是经典范围内，从上述介绍中我们依然可以体会到满满的套路感。除此之外，还有一些并不按照常规套路出牌，它们同样也表达了位置编码。

CNN式

尽管经典的将CNN用于NLP的工作《Convolutional Sequence to Sequence Learning》往里边加入了位置编码，但我们知道一般的CNN模型尤其是图像中的CNN模型，都是没有另外加位置编码的，那CNN模型究竟是怎么捕捉位置信息的呢？

如果让笔者来回答，那么答案可能是卷积核的各项异性导致了它能分辨出不同方向的相对位置。不过ICLR 2020的论文《How Much Position Information Do Convolutional Neural Networks Encode?》给出了一个可能让人比较意外的答案：CNN模型的位置信息，是Zero Padding泄漏的！

我们知道，为了使得卷积编码过程中的feature保持一定的大小，我们通常会对输入padding一定的0，而这篇论文显示该操作导致模型有能力识别位置信息。也就是说，卷积核的各向异性固然重要，但是最根本的是zero padding的存在，那么可以想象，实际上提取的是当前位置与padding的边界的相对距离。

不过，这个能力依赖于CNN的局部性，像Attention这种全局的无先验结构并不适用，如果只关心Transformer位置编码方案的读者，这就权当是扩展一下视野吧。

复数式

复数式位置编码可谓是最特立独行的一种位置编码方案了，它来自ICLR 2020的论文《Encoding word order in complex embeddings》。论文的主要思想是结合复数的性质以及一些基本原理，推导出了它的位置编码形式（Complex Order）为：

$$\left[r_{j,1} e^{i(\omega_{j,1} k + \theta_{j,1})}, \dots, r_{j,2} e^{i(\omega_{j,2} k + \theta_{j,2})}, \dots, r_{j,d} e^{i(\omega_{j,d} k + \theta_{j,d})} \right] \quad (12)$$

这里的 i 是虚数单位， j 代表某个词， k 代表该词所在的位置，而

$$\begin{aligned} \mathbf{r}_j &= [r_{j,1}, r_{j,2}, \dots, r_{j,d}] \\ \boldsymbol{\omega}_j &= [\omega_{j,1}, \omega_{j,2}, \dots, \omega_{j,d}] \\ \boldsymbol{\theta}_j &= [\theta_{j,1}, \theta_{j,2}, \dots, \theta_{j,d}] \end{aligned} \quad (13)$$

代表词 j 的三组词向量。你没看错，它确实假设每个词有三组跟位置无关的词向量了（当然可以按照某种形式进行参数共享，使得它退化为两组甚至一组），然后跟位置 k 相关的词向量就按照上述公式运算。

你以为引入多组词向量就是它最特立独行的地方了？并不是！我们看到式(12)还是复数形式，你猜它接下来怎么着？将它实数化？非也，它是将它直接用于复数模型！也就是说，它走的是一条复数模型路线，不仅仅输入的Embedding层是复数的，里边的每一层Transformer都是复数的，它还实现和对比了复数版的Fasttext、LSTM、CNN等模型！这篇文章的一作是Benyou Wang，可以搜到他的相关工作基本上都是围绕着复数模型展开的，可谓复数模型的铁杆粉了～

融合式

无偶独有，利用复数的形式，笔者其实也构思了一种比较巧的位置编码，它可以将绝对位置编码与相对位置编码融于一体，分享在此，有兴趣的读者欢迎一起交流研究。

简单起见，我们先假设 $\mathbf{q}_m, \mathbf{k}_n$ 是所在位置分别为 m, n 的二维行向量，既然是二维，那么我们可以将它当作复数来运算。我们知道，Attention关键之处在于向量的内积，用复数表示为

$$\langle \mathbf{q}_m, \mathbf{k}_n \rangle = \text{Re} [\mathbf{q}_m \mathbf{k}_n^*] \quad (14)$$

其中 $*$ 是共轭复数，右端的乘法是普通的复数乘法， $\text{Re}[\]$ 表示取结果的实部。上式也就是说：

两个二维向量的内积，等于把它们当复数看时，一个复数与另一个复数的共轭的乘积实部。

如果我们将 $\mathbf{q}_m, \mathbf{k}_n$ 分别乘以 $e^{im\theta}, e^{in\theta}$ 变成 $\mathbf{q}_m e^{im\theta}, \mathbf{k}_n e^{in\theta}$ ，那么就相当于给它们配上了绝对位置编码了（因为显式地依赖绝对位置 m, n ），然后放到内积里边，我们有

$$\langle \mathbf{q}_m e^{im\theta}, \mathbf{k}_n e^{in\theta} \rangle = \text{Re} \left[(\mathbf{q}_m e^{im\theta}) (\mathbf{k}_n e^{in\theta})^* \right] = \text{Re} \left[\mathbf{q}_m \mathbf{k}_n^* e^{i(m-n)\theta} \right] \quad (15)$$

相当有意思的是，内积只依赖于相对位置 $m - n$ ！这就巧妙地将绝对位置与相对位置融合在一起了。

注意，我们没有像Complex Order那么“疯狂”，上述运算本质上还是在实数范畴内的，只不过我们是借助复数来完成了某些推导而已。由上述结果可知，对于位置为 n 的二维实数向量 $[x, y]$ ，我们当它复数来运算，乘以 $e^{in\theta}$ ，得到恒等式

$$(x + yi)e^{in\theta} = (x \cos n\theta - y \sin n\theta) + i(x \sin n\theta + y \cos n\theta) \quad (16)$$

这也就是意味着，通过

$$\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{pmatrix} x \cos n\theta - y \sin n\theta \\ x \sin n\theta + y \cos n\theta \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} \cos n\theta + \begin{pmatrix} -y \\ x \end{pmatrix} \sin n\theta \quad (17)$$

来赋予 $[x, y]$ 绝对位置信息，那么在Attention运算的时候也等价于相对位置编码。如果是多于二维的向量，可以考虑每二维为一组进行同样的运算，每一组的 θ 可以不一样。

这样一来，我们得到了一种融绝对位置与相对位置于一体的位置编码方案，从形式上看它有点像乘性的绝对位置编码，通过在 $\mathbf{q}, \mathbf{k}$ 中施行该位置编码，那么效果就等价于相对位置编码，而如果还需要显式的绝对位置信息，则可以同时在 $\mathbf{v}$ 上也施行这种位置编码。总的来说，我们通过绝对位置的操作，可以达到绝对位置的效果，也能达到相对位置的效果，初步实验显示它是可以work的，但还没有充分验证，欢迎大家尝试交流。

文章内容小结

本文汇总了一些位置编码的工作，大体分为绝对式、相对式、非套路式三种，从中我们可以看到各种神奇的操作。最后，笔者分享了自己构思的一种融合绝对位置与相对位置的编码方案，供有兴趣的读者参考。

转载到请包括本文地址：<https://spaces.ac.cn/archives/8130>

更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Feb. 03, 2021). 《让研究人员绞尽脑汁的Transformer位置编码》 [Blog post]. Retrieved from <https://spaces.ac.cn/archives/8130>

```
@online{kexuefm-8130,  
  title={让研究人员绞尽脑汁的Transformer位置编码},  
  author={苏剑林},  
  year={2021},  
  month={Feb},  
  url={\url{https://spaces.ac.cn/archives/8130}},  
}
```